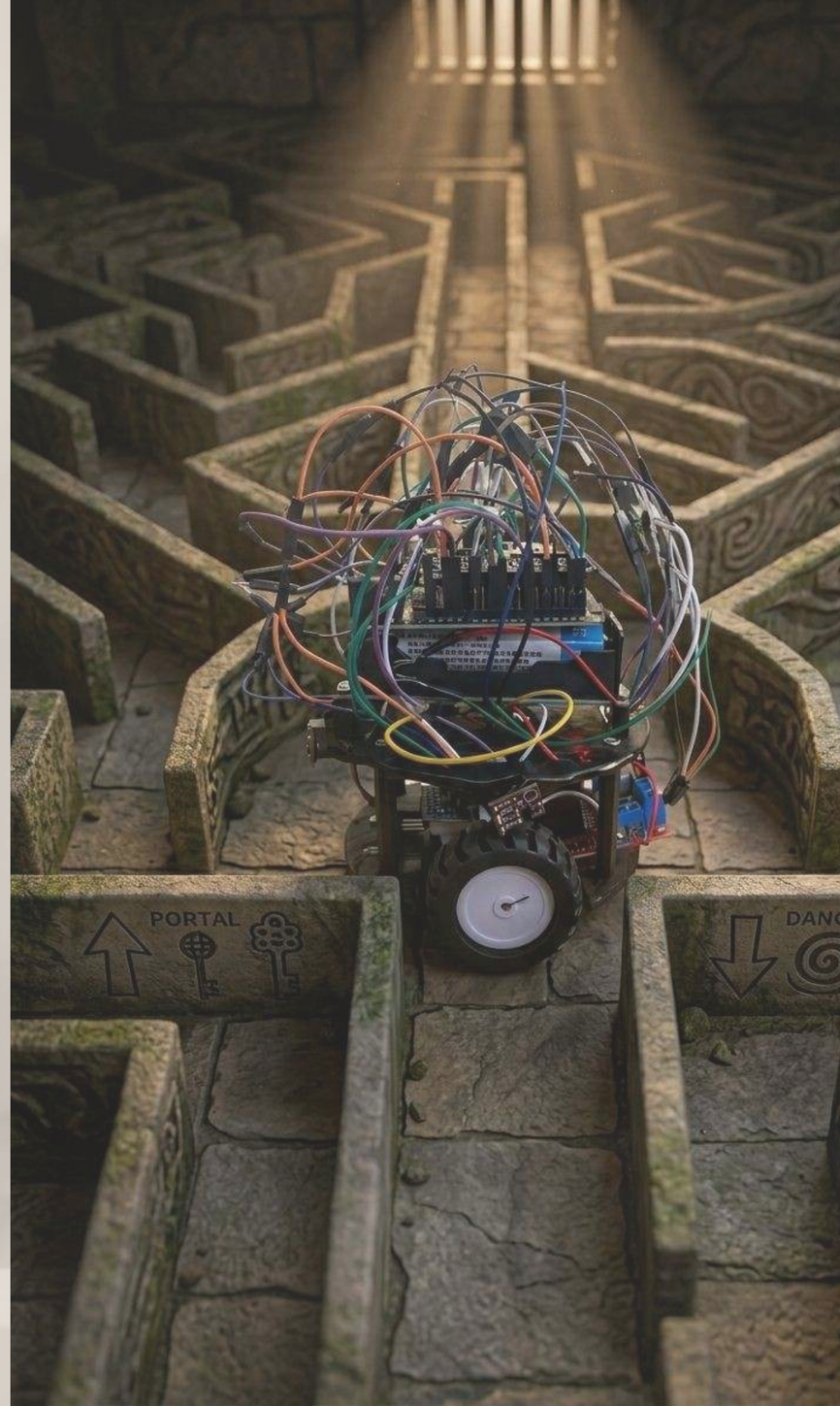
SHAMSHOON

MICRO MAZE ROBOT



Our team

Instructor Dr.Wasel

Waad ziadeh

Labiba sharia

Joud thaher

layan salem

Component Overview & Purpose

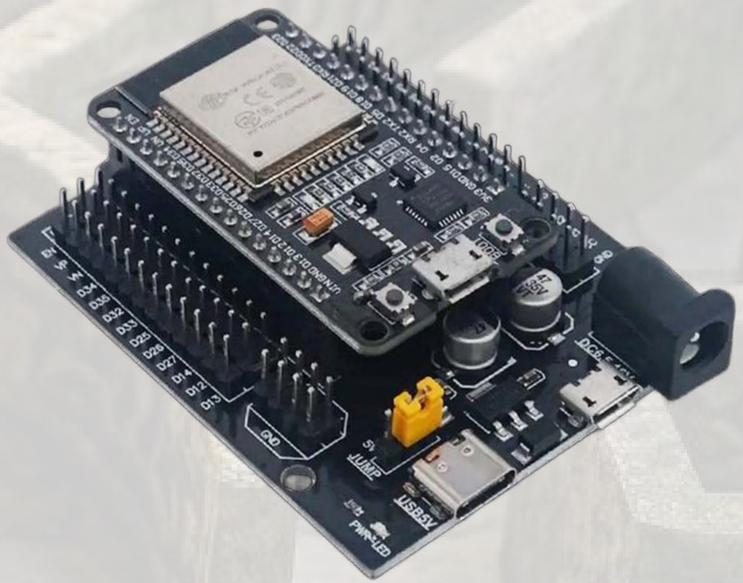
7 | Components

3 | Sensors

2 | Motors

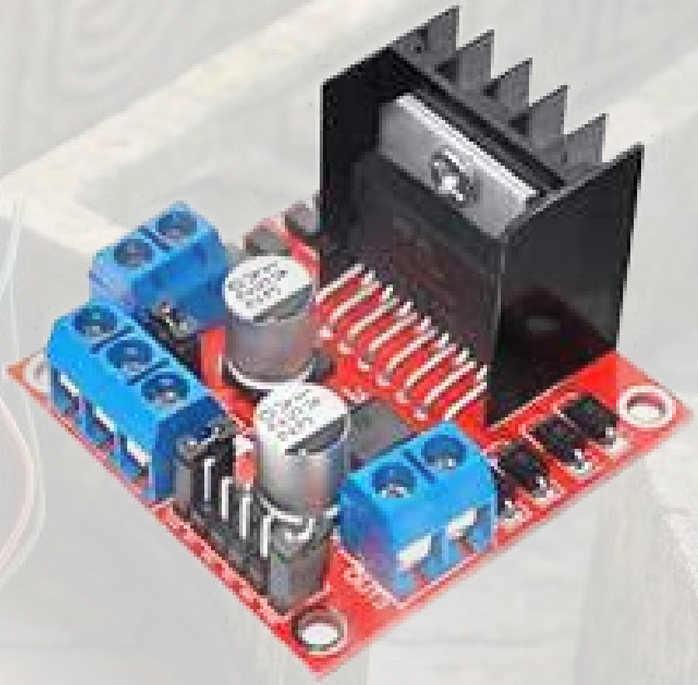
1 | ESP32 + Shield Microcontroller

- Acts as the brain of the robot, handling all processing and decision-making
- Dual-core processor handles multiple tasks simultaneously (reading sensors + controlling motors
- Large number of GPIO pins to connect all sensors and peripherals
- The shield provides clean and organized wiring with stable power distribution



2 | L298N Motor Driver (H bridge)

- GPIO pins cannot supply enough current to drive motors directly, so the L298N acts as a power bridge
- Controls speed of motors using PWM signals from the ESP32
- Controls direction (forward/backward) of each motor independently
- Dual channel allows both motors to be controlled from a single driver mod

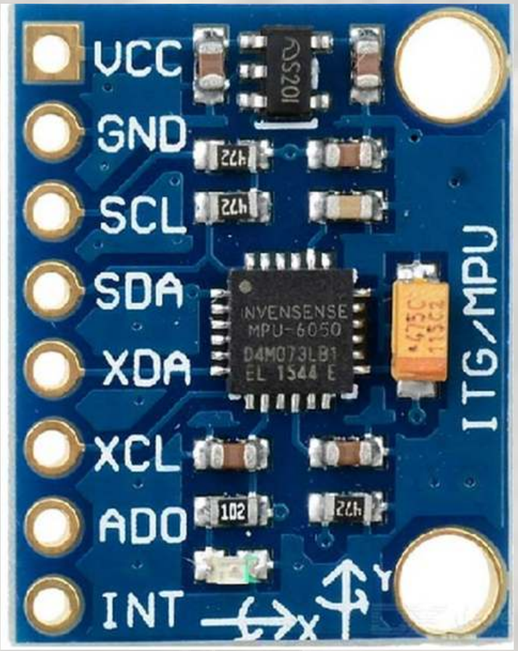


3 | LIDAR SENSORS ×3

- Measures distance using laser pulses with high accuracy
- Three sensors cover front, left, and right walls simultaneously
- Gives the robot 360° spatial awareness inside the maze
- Enables real-time decisions like turning, stopping, or going straight
- More reliable than ultrasonic sensors in terms of speed and precision

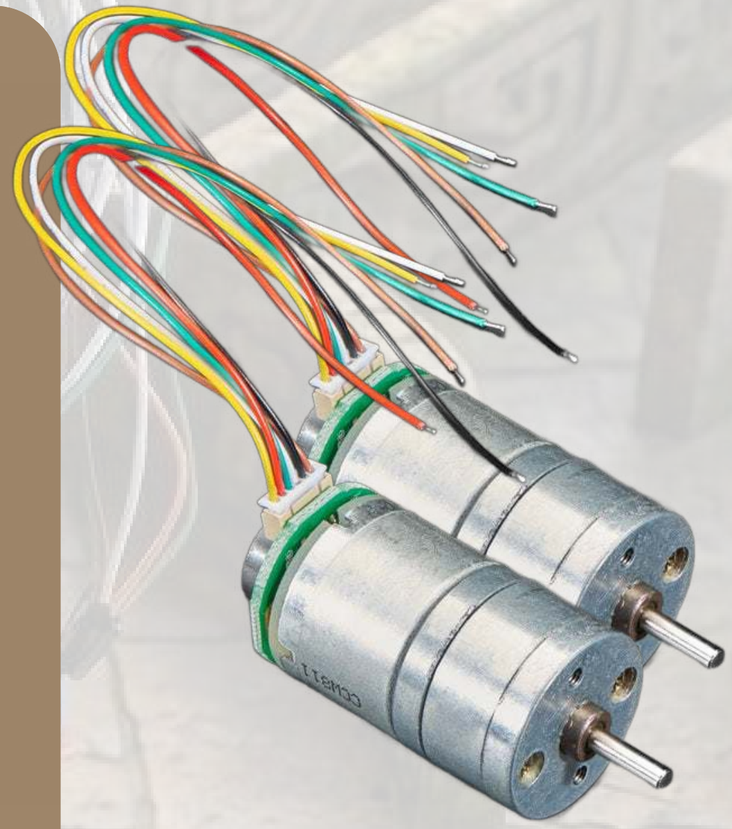
4 | MPU-6050 (Accelerometer + Gyroscope)

- 6-axis IMU combining 3-axis accelerometer and 3-axis gyroscope
- Detects the robot's orientation, tilt, and rotation angle
- Confirms accurate 90° turns when navigating maze corners
- Corrects errors caused by wheel slippage that encoders may miss
- Improves overall navigation accuracy and stability
- Works alongside encoders to provide more reliable position tracking



5 | 2 DC Motors with Encoders

- Provide the differential drive system for movement and steering
- Encoders send pulse feedback to the ESP32 to measure exact wheel rotation
- Enables odometry — calculating distance traveled and position
- Ensures both wheels spin at equal speed for straight-line motion
- Helps detect and correct slippage or drift during navigation

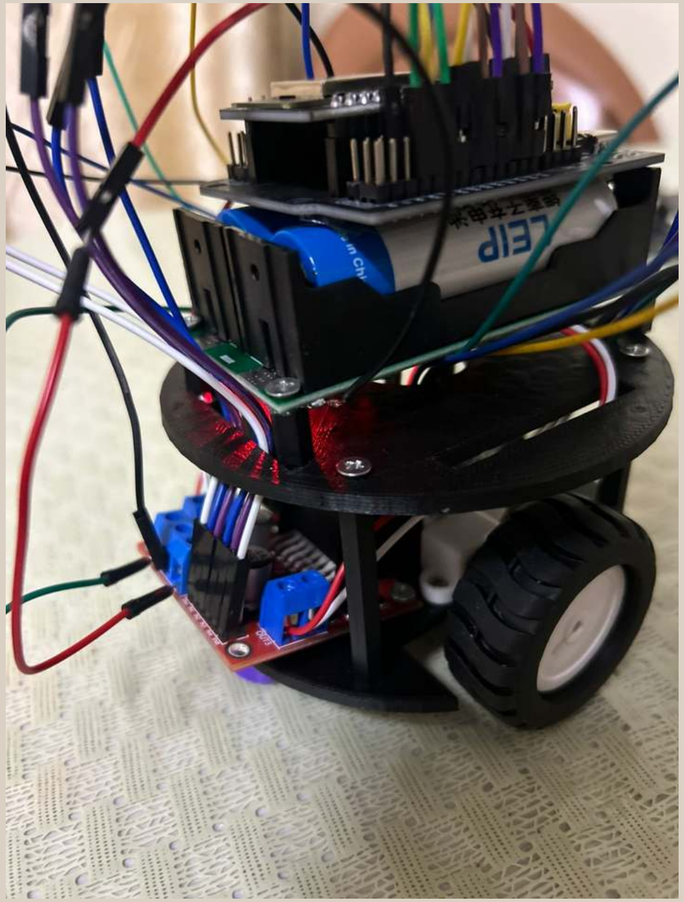


6 | Wheels + Caster Wheel

- Two driven wheels handle forward, backward, and turning movement
- Caster wheel provides balance and stability without extra motor load
- Reduces friction and allows smooth direction changes
- Supports the robot's weight evenly on flat maze surfaces

7 | Lithium-Ion Battery+ Buck-Boost Converter

- Lithium-ion chosen for high energy density and rechargeable capability
- Delivers stable current needed by motors and electronics
- Buck-boost converter regulates output voltage as battery drains
- Protects sensitive components (ESP32, sensors) from voltage fluctuations
- Ensures the robot runs consistently from full charge to low battery



Robot Design

We designed the robot body using 3D printing to be compact and lightweight in order to improve its overall performance.

Design Decision: Small and Lightweight Structure

To allow smooth movement inside the maze

To allow smooth movement inside the mazTo improve maneuverability in narrow pathse

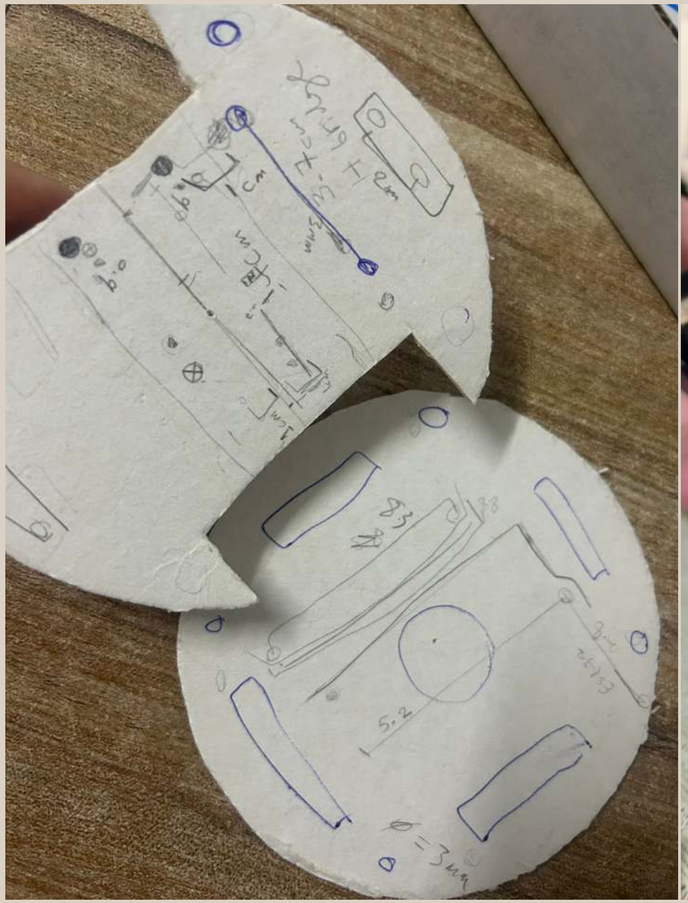
Less mass → higher speed

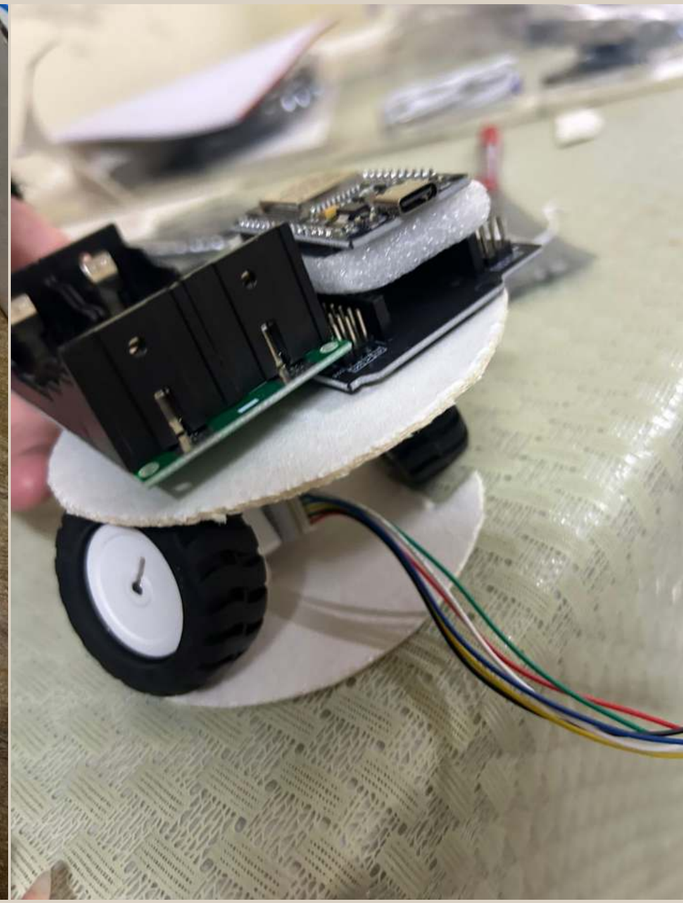
Reduced load on the motors

A lighter robot requires less torque, which increases acceleration and improves response time.

Robot Body Design and Development

Before printing the final body, we first created a cardboard prototype to take accurate measurements and test component placement.
This step helped us determine the correct dimensions and avoid material waste before 3D printing.





Design Choice: 3D Printing Technology

High precision manufacturig

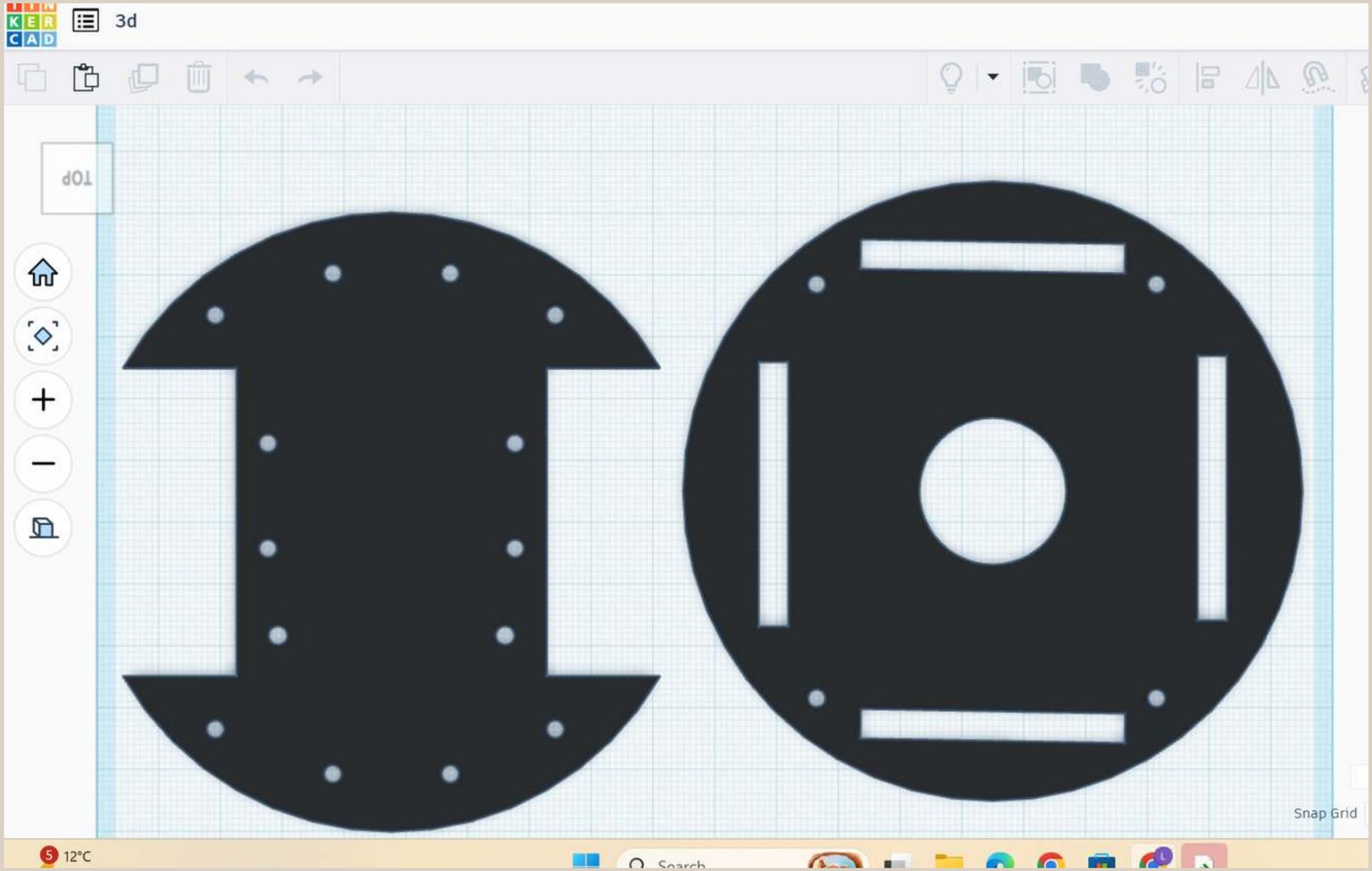
Ability to customize the design

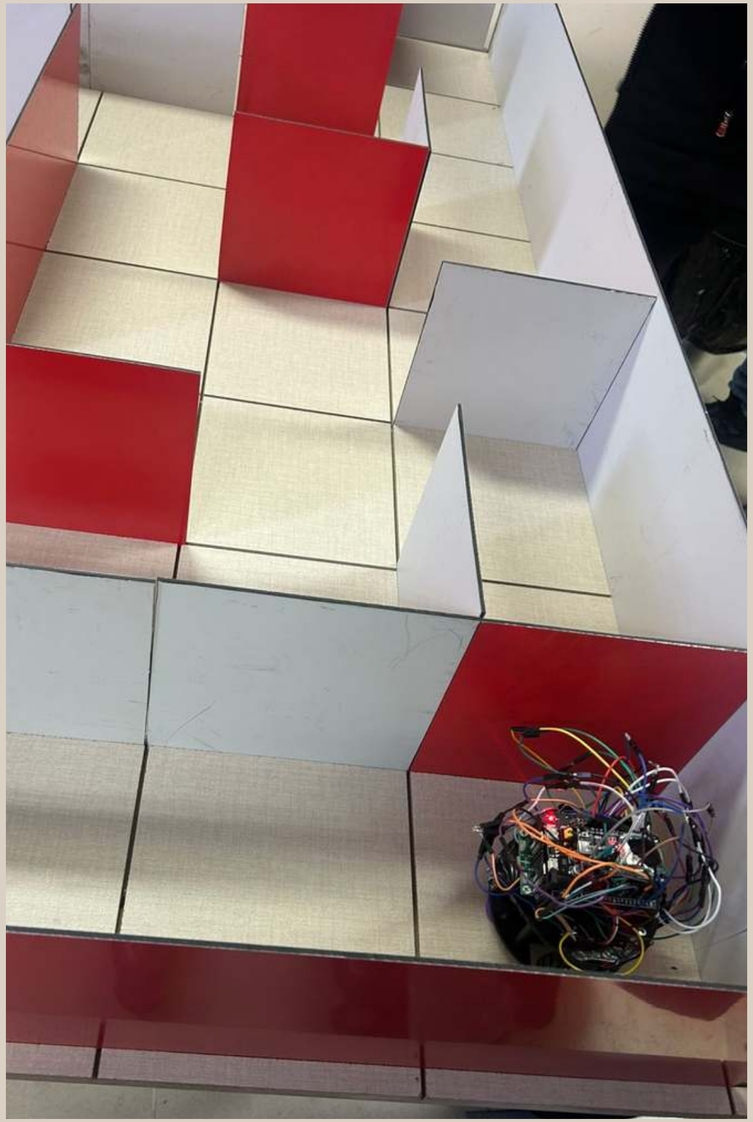
Modern manufacturing approach

Lightweight structure

We used 3D printing to gain practical experience with advanced manufacturing technology, which is considered an important tool in the future of engineering and product development. As Dr. Wasel mentioned, 3D printing represents a key part of future innovation and rapid prototyping.

Design Choice: 3D Printing Technology





State Space Modelling

Track robot position ([x, y, θ])

Predict motion from control inputs

Provide position estimation for maze navigation

State Definition

State vector:

- $x=[x, y, \theta]$

Where:

- $x \rightarrow$ East/West position
- $y \rightarrow$ North/South position
- $\theta \rightarrow$ robot heading

Coordinate system:

- 0°= North
- 90°= East
- Clockwise positive

State Update Model

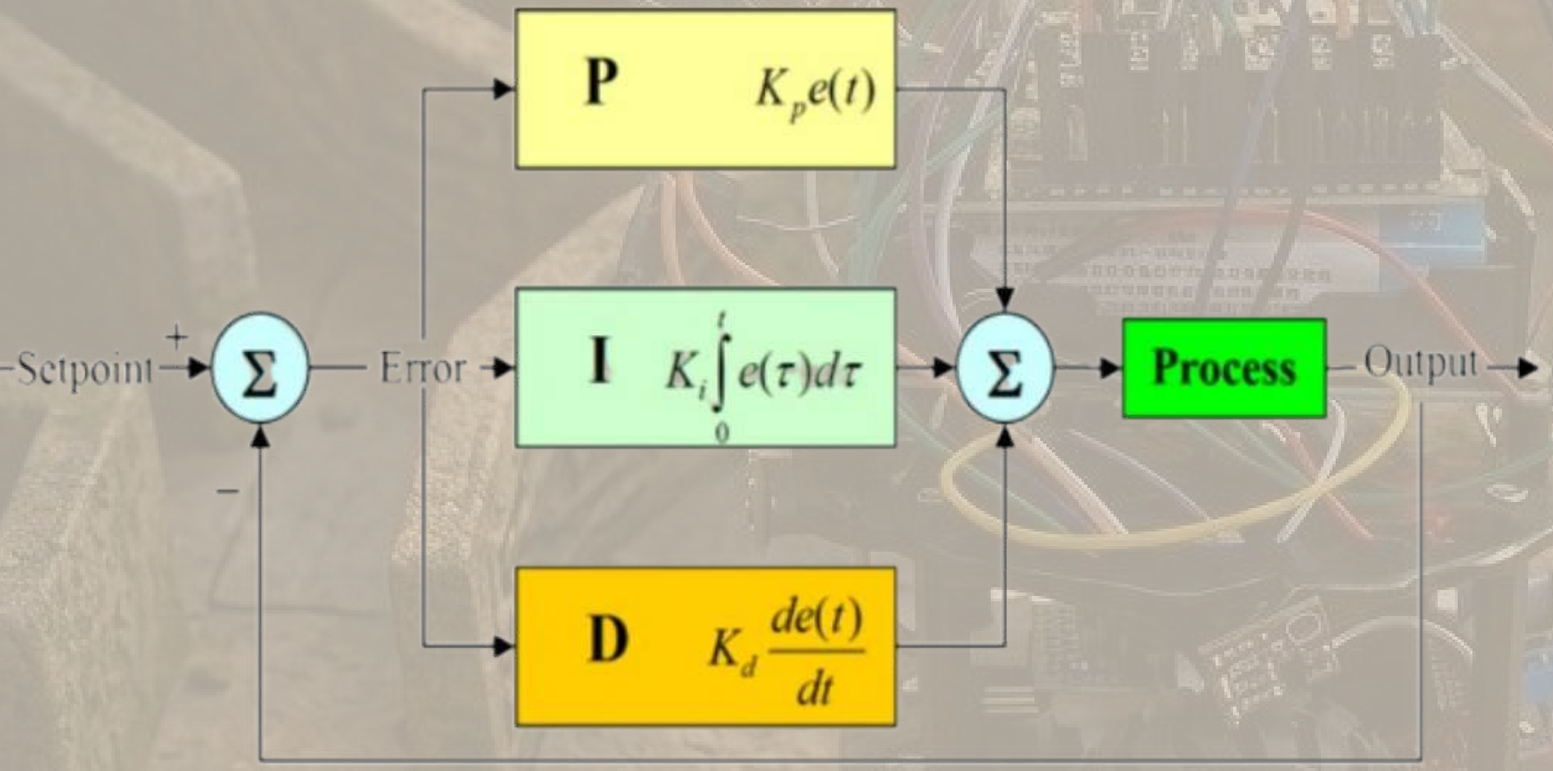
- $x_{new} = Ax + Bu$
- $A = I$ (identity)
- $u = [v, \omega]$
  - linear velocity from encoders
  - angular velocity from gyro

This is a kinematic differential-drive model.

State Validation Tests

- we tested the following:
- Move forward → y increases by 0.2 m
- Rotate 90° → θ=90∘
- Move forward again → x increases
- This verified:
- correct coordinate transformation
- correct heading-based motion

PID Control



Maintain constant speed

Achieve accurate 90° turns

Prevent drift and oscillation

Proportional–Integral–Derivative controller

Adjusts motor power to minimize error between:
Target speed and Measured speed

Error:

e = target speed - measured speed (from encoders)

Control law:

$$u = K_p e + K_i \int e dt + K_d \frac{de}{dt}$$

Control Loop Flow

We use one PID per wheel:

- Input: Target wheel velocity
- Feedback: Encoder velocity
- Output: Motor PWM (0–255)
- Update time: 20 ms control loop

Target speed → PID → Motor PWM → Encoder → measured speed → PID

Motion Primitives Using PID

PID enables:

- Constant speed forward motion
- Accurate cell-to-cell movement
- Stable 90° turns (with gyro feedback)

Used i MotionController

Manual PID Tuning

1. Start with P only
2. Add I to remove steady-state error
3. Add D to reduce oscillation

Final gains chosen experimentally.

FloodFill Algorithm

8×8 grid (64 cells) maze

Each cell stores:

- 4 walls (N,E,S,W) → 4-bit encoding
- distance value

Goal: center 4 cells

Algorithm

Steps:

1. Initialize goal distance = 0
2. Set others = ∞
3. BFS flood from goal
4. Each cell gets minimum distance

Robot always moves to lowest neighbor.

Wall Detection

Using:

- LiDAR sensors (front, left, right)
- Convert relative walls → absolute direction using heading θ
- Update maze map in real time

Path Decision

For current cell:

- Check 4 neighbors
- Ignore blocked by wall
- Choose minimum distance

Output:

next direction command

Full System Flow

FloodFill → next cell →
MotionController → PID speed control → Motors →
Encoders + MPU → StateSpace → updated position
→ repeat